AUTHOR'S NAME

TITLE

Bachelor's/Master's Thesis

to achieve the university degree of

Bachelor/Master of Science

Bachelor's/Master's degree program: Author's program

submitted to

Graz University of Technology

Supervisor

Supervisor's name

Electric Drives and Power Electronic Systems Institute

Graz, Month XX, Year

Acknowledgements

This section is your opportunity to thank those who have helped and supported you personally and professionally during your thesis or dissertation process.

Abstract

An abstract is a short summary of a longer work. The abstract concisely reports the aims and outcomes of your research so that readers know exactly what your paper is about.

Kurzfassung

Eine Kurzfassung ist eine kurze Zusammenfassung deiner Arbeit. Die Kurzfassung berichtet prägnant über die Ziele und Ergebnisse deiner Forschung, sodass Leser genau wissen worum es in der Arbeit geht.

Contents

1 Introduction

This chapter gives you an overview of the most important functions you will need during the creation of your thesis. There will be a lot of examples on how to insert figures, write mathematical expressions, create references, etc.
In case you need a more detailed explanation on several LaTeX commands, we advise you to check out Overleaf's documentation of LaTeX at [1].

1.1 Document structure

Your thesis has to be logically divided into chapters, sections, sub-sections, etc.
To ensure that all chapters and sections are correctly displayed in the table of contents, you need to label them accordingly, using commands like `\chapter{}`, `\section{}`, `\subsection{}`, etc.

1.1.1 Hierarchical structure

In LaTeX, the hierarchical structure of a document is organized using chapters, sections, subsections, and subsubsections. The most relevant commands and their usage are explained below.

- `\chapter{chapter title}`: Top-level sectioning command. Typically used for main chapters in a thesis.
- `\section{section title}`: Subdivision of a chapter. Used for major sections within a chapter.
- `\subsection{subsection title}`: Subdivision of a section. Used for smaller sections within a major section.
- `\subsubsection{subsubsection title}`: Subdivision of a subsection. Used for detailed subsections within a subsection.

1.1.2 Labeling and Cross-Referencing

To label chapters, sections, subsections, and subsubsections for cross-referencing, you use the `\label{}` command. You can then reference these labels anywhere in your document using the `\ref{}` or `\pageref{}` commands.

Labeling

Here is an example of how to label chapters or sections:

```
\chapter{Introduction}
\label{ch:introduction}

\section{Inserting Figures, Subplots, etc.}
\textbackslash label{sec:insert-figures}

\subsection{Inserting Two Figures Stacked Vertically}
\label{subsec:insert-stacked}
```

Cross-referencing

Here is an example of how to reference the labeled chapters or sections in the text:

```
As described in section~\ref{ch:introduction}, we are describing how
to insert figures in section~\ref{sec:insert-figures}.
The explanation on how to vertically stack two figures
can be found on pg.~\pageref{subsec:insert-stacked}.
```

When compiled the text from above looks like this in the PDF:

As described in section 1, we describe how to insert figures in section 1.2. The explanation on how to vertically stack two figures can be found on pg. 5.

1.2 Inserting Figures, Subplots, etc.

To insert a figure in LaTeX, we need to include the `graphicx` package. This package is already included in our `packages.tex` file. In the following sections, you will learn how to insert figures in different ways.

1.2.1 Inserting a Single Figure

You can use the `figure` environment with the `\includegraphics` command to insert a single figure. Here's an example:

```
\begin{figure}[htbp]
    \centering
    \includegraphics[width=0.5\textwidth]{path/to/your/image.png}
    \caption{Example figure}
    \label{fig:single}
\end{figure}
```

Figure 1.1: Example figure

Explanation of the individual commands:

- `\beginfigure`: Creates figure enviorment.
- `[htbp]`: Placement specifier (here, top, bottom, page).
- `\centering`: Centers the figure.
- `\includegraphics[width=0.5\textwidth]{path/to/your/image.png}`: Adds the image which ist stored at *path/to/your/image.png* and adjusts the width of the image to 50% of the text width.
- `\caption{}`: Adds a caption below the figure.
- `\label{}`: Adds a label for referencing the figure.

This LaTeX-code will insert a single figure with the caption 'Example figure' and a label for cross-referencing. You can reference a figure using the `\ref{fig:label}` command, where you insert the actual label you gave the figure you want to reference. In Fig. 1.1, you can see how the code from above will look when applied in LaTeX.

1.2.2 Inserting Two Figures Side by Side

You can use the `subfigure` package to insert two figures side by side. Here's an example:

```
\begin{figure}[htbp]
    \centering
    \begin{subfigure}[b]{0.4\textwidth}
        \includegraphics[width=\textwidth]{path/to/your/image-a.png}
        \caption{Figure A}
        \label{fig:subfiga}
    \end{subfigure}
    \hspace{0.05\textwidth}
    \begin{subfigure}[b]{0.4\textwidth}
        \includegraphics[width=\textwidth]{path/to/your/image-b.png}
        \caption{Figure B}
        \label{fig:subfigb}
    \end{subfigure}
    \caption{Two figures side by side}
    \label{fig:sidebyside}
\end{figure}
```

(a) Figure A (b) Figure B

Figure 1.2: Two figures side by side

Explanation:

- `\begin{subfigure}[b]{0.4\textwidth}`: Creates a subfigure occupying 40% of the text width.

- `\hspace{0.05\textwidth}`: Adds horizontal space of 5% of the textwidth between the subfigures.

This code will insert two figures side by side with captions 'Figure A' and 'Figure B' and a common caption 'Two figures side by side'.

1.2.3 Inserting Two Figures Stacked Vertically

To insert two figures stacked vertically, you can use the `subfigure` package similarly to the previous example. Here's an example:

```
\begin{figure}[htbp]
    \centering
    \begin{subfigure}[b]{0.5\textwidth}
        \includegraphics[width=0.8\textwidth]{path/to/your/image-a.png}
        \caption{Subfigure A}
        \label{fig:subfigva}
    \end{subfigure}
    \vspace{0.5cm}
    \begin{subfigure}[b]{0.5\textwidth}
        \includegraphics[width=0.8\textwidth]{path/to/your/image-b.png}
        \caption{Subfigure B}
        \label{fig:subfigvb}
    \end{subfigure}
    \caption{Two figures stacked vertically}
    \label{fig:stacked}
\end{figure}
```

Explanation:

- `\vspace{0.5cm}`: Adds vertical space (of 0.5cm) between the subfigures.

This will insert two figures stacked vertically with captions 'Figure A' and 'Figure B' and a common caption 'Two figures stacked vertically'. You can reference each subfigure by itself by using its labels or you can reference both figures at once by using their common label. In Fig. 1.3a, you can see the EALS logo in color, while Fig. 1.3b shows the EALS logo in black. Both figures are included in Fig. 1.3.

1.2.4 Including List of Figures

To create a list of figures use the **listoffigures** command. The caption of each labeled figure will be used to generate this list.

(a) Subfigure A

(b) Subfigure B

Figure 1.3: Two figures stacked vertically

1.3 Tables

To create a simple table, use the `table` as well as the `tabular` environment.

1.3.1 Basic Table Creation

The code example below, shows how to generate a simple table. Here the structure of your commands is important.

```
\begin{table}[h]
    \centering
    \begin{tabular}{ccc}
    \hline
    Header 1 & Header 2 & Header 3 \\
    \hline
    Row 1, Col 1 & Row 1, Col 2 & Row 1, Col 3 \\
    Row 2, Col 1 & Row 2, Col 2 & Row 2, Col 3 \\
    \hline
    \end{tabular}
    \caption{A simple table}
    \label{tab:simple}
\end{table}
```

Explanation of the individual commands:

Header 1	Header 2	Header 3
Row 1, Col 1	Row 1, Col 2	Row 1, Col 3
Row 2, Col 1	Row 2, Col 2	Row 2, Col 3

Table 1.1: A simple table

- **begintable[h]**: Creates a table environment. [h] specifies that the table should be placed "here" in the file.
- **centering**: Centers the table.
- **hline**: Inserts a horizontal line in-between the rows.
- **&**: Separates columns within a row.
- \\: Ends the current row.
- **caption{...}**: Adds a caption to the table.
- **label{...}**: Adds a label for referencing the table with \ref{}.

1.3.2 Table Formatting

You can adjust column alignment and add more complex formatting. This can be done by adding an alignment specifier and vertical bars | for vertical lines to the tabular command, as can be seen in the code example below.

```
\begin{table}[h]
\centering
\begin{tabular}{l|c|r}
Left-aligned & Center-aligned & Right-aligned \\
\hline
Data 1 & Data 2 & Data 3 \\
Data 4 & Data 5 & Data 6 \\
\end{tabular}
\caption{Formatted table}
\label{tab:formatted}
\end{table}
```

Left-aligned	Center-aligned	Right-aligned
Data 1	Data 2	Data 3
Data 4	Data 5	Data 6

Table 1.2: Formatted table

Explanation:

- **l**: Left-aligns the column.
- **c**: Center-aligns the column.
- **r**: Right-aligns the column.
- |: Adds vertical line.

1.3.3 Adding Multi-row and Multi-column

You can also add multirows or multicolumns to your table by using the commands seen in the code example below.

```
\begin{table}[h]
    \centering
    \begin{tabular}{|c|c|c|}
    \hline
    \multirow{2}{*}{Multi-row} & Col 1 & Col 2 \\
    & Col 3 & Col 4 \\
    \hline
    Col 1 & \multicolumn{2}{|c|}{Multi-column} \\
    \hline
    \end{tabular}
\caption{Table with multi-row and multi-column}
\label{tab:multi}
\end{table}
```

This is what the code example from above looks like when compiled:

Multi-row	Col 1 Col 3	Col 2 Col 4
Col 1	Multi-column	

Table 1.3: Table with multi-row and multi-column

Explanation:

- **usepackagemultirow**: Import the multirow package.
- **multirow{2}{*}{Multi-row}**: Spans the content 'Multi-row' over 2 rows. The * denotes the natural width.
- **multicolumn{2}{|c|}{Multi-column}**: Spans the content 'Multi-column' over 2 columns, with centered alignment and vertical lines on either side.

1.3.4 Including List of Tables

To create a list of tables use the **listoftables** command. The caption of each labeled table will be used to generate this list.

1.4 Mathematics

When writing mathematical symbols or expressions, you need to do so in a mathematical environment. For this, you need the*amsmath* package and its offered commands.
To display a mathematical expression correctly you need to wrap them in a mathematical environment. This can be established in several ways. Here are some ways with which you can enable the so-called *Math mode*:

- $....$ or \(....\): These commands are used for in-line math (i.e. writing mathemtical symbols or expressions). They are used when including mathematical expressions in flow text. Both commands achieve the same result.

  Example:

  > To calculate the parameter λ we need to determine the circular-frequency ω and phase shift φ.'

- \[....\]: This command is used for writing equations without numbering/labeling them. It goes into a new line and centers the equation (it's not centred here because we're in an itemise environment, but in normal text it is.).

  Example:

  > 'The mass-energy equivalence is stated by the equation
  >
  > $$E = mc^2,$$
  >
  > which was discovered by Albert Einstein.'

- \begin{equation} ... \end{equation}: This command is used for equations. It goes to a newline and centers equations with label.

  Example:

  $$\frac{\mathrm{d}y}{\mathrm{d}x} = \frac{\mathrm{d}y}{\mathrm{d}g} \cdot \frac{\mathrm{d}g}{\mathrm{d}x} \tag{1.1}$$

  This environment automatically numbers your equations in chronological order. If you want your equation to not be numbered, use equation* (with an asterisk) otherwise.

  Example:

  $$\frac{\mathrm{d}y}{\mathrm{d}x} = \frac{\mathrm{d}y}{\mathrm{d}g} \cdot \frac{\mathrm{d}g}{\mathrm{d}x}$$

- \begin{align} ... \end{align}: This command is used if there are several equations that you need to align vertically. It goes to a new line and centers equations.

  Example:

  Our calculations yield the following system of equations:

  $$2x - 5y = 8 \tag{1.2}$$
  $$3x + 9y = -12 \tag{1.3}$$
  $$6x + 4y = 45 \tag{1.4}$$

  Like the *equation*-environment, the *align*-environment automatically numbers each line of your equations. If you want your equations not to be numbered, use equation* (with an asterisk) otherwise.

- \begin{multline} ... \end{multline}: This command is used, for equations that are longer than a line. Insert a double backslash to set a point for the equation to be broken. The first part will be aligned to the left and the second part will be displayed in the next line and aligned to the right.

  Again, the use of an asterisk * in the environment name determines whether the equation is numbered or not.

  Example:

  $$p(x) = 3x^6 + 14x^5y + 590x^4y^2 + 19x^3y^3 \\ - 12x^2y^4 - 12xy^5 + 2y^6 - a^3b^3 \tag{1.5}$$

1.4.1 Macros

To make the writing of equations faster and easier, this template provides several macros which are shortened versions of frequently used mathematical LaTeX functions. The definition of these macros can be found in the `macros.tex` file. Feel free to add your own macros for e.g. defining variables that are used multiple times throughout the document.

\mrm{}

To avoid text in a mathematical env to become *cursive*, use the macro-command `\mrm{}` which we provide in this template (see `macros.tex`).
Without `\mrm{}`:

$$V_{pp} = 6V$$

With `\mrm{}`:

$$V_{\mathrm{pp}} = 6\mathrm{V}$$

This should be remembered because only variables should be written in cursive. In this case, neither the subscript "pp" or the unit V is a variable, so it shouldn't be cursive. This macro is a short version of the LaTeX function `\mathremove{}`.

\diff

When writing a derivation we differentiate between a full derivation and a partial derivation. The full derivation is denoted with the letter d. This can be written in a math environment by using the macro `\diff`.
For example:

$$\frac{\mathrm{d}Q}{\mathrm{d}t} = \frac{\mathrm{d}s}{\mathrm{d}t}$$

The partial derivation is denoted with the symbol ∂. by using the LaTeX function `\partial`.
For example:

$$\frac{\partial Q}{\partial t} = \frac{\partial s}{\partial t}$$

\RE, \IM

To denote the real-valued part of a complex-valued variable, the symbol $\mathfrak{Re}$ is used. This can be written in a math environment by using the macro `\RE`. To denote the imaginary-valued part of a complex-valued variable, the symbol $\mathfrak{Im}$ is used. This can be written in a math environment by using the macro `\IM`.

Example:

$$|z| = \sqrt{\mathfrak{Re}(z)^2 + \mathfrak{Im}(z)^2}$$

\J

The imaginary unit is either denoted by the symbol j or i. In electrical engineering, j is more common. Because j is not a variable, it is not written in cursive. The macro `/J` enables us to use j in a mathematical environment.
Example:

$$z = a + \mathrm{j}b$$

\T

A transposed vector is denoted with the letter T in superscript. To write this in a mathematical environment the macro `\T` can be used.
Example:

$$(A + B)^{\mathrm{T}} = A^{\mathrm{T}} + B^{\mathrm{T}}$$

\COS,\SIN

To add cos and sin with large brackets in a mathematical environment, we can use the `\COS{}` or `\SIN{}` macro.

$$\cos\left(\frac{\omega}{\omega_0}\right)$$

1.5 Code listings

1.5.1 Inline Code

To include short snippets of code within your text, you can use the `\texttt{}`.
The following example shows how to include an inline code example.

Code in LaTeX: \texttt{print('Hello, World!')}
Output in PDF: `print("Hello, World!")`

The `\texttt{}` command formats the enclosed text in a typewriter (monospaced) font, suitable for inline code.

1.5.2 Code Blocks

For larger blocks of code, you can use the `verbatim` environment.

```
\begin{verbatim}
def hello_world():
    print("Hello, World!")
\textbackslash end{verbatim}
```

This environment displays the enclosed text exactly as it is, preserving whitespace and formatting.

1.5.3 Highlighted Code with listings

For syntax highlighting, you can use the `listings` package:

```
\lstset{language=Python}
\begin{lstlisting}
def hello_world():
    print("Hello, World!")
\end{lstlisting}
```

Explanation:

- **\usepackage{listings}**: Imports the listings package.
- **\lstset{language=Python}**: Sets the language for syntax highlighting (Python in this example).
- **\begin{lstlisting}...\end{lstlisting}**: Creates a code block with syntax highlighting.

1.5.4 Customizing listings

You can customize the appearance of your code with various options, which are listed in the LaTeX code example below.

```
\usepackage{listings}
\usepackage{xcolor}
\lstset{
    language=Python,
    basicstyle=\ttfamily\footnotesize,
    keywordstyle=\color{blue},
    stringstyle=\color{red},
    commentstyle=\color{green},
    backgroundcolor=\color{lightgray},
    numbers=left,
    numberstyle=\tiny\color{gray},
    stepnumber=1,
    frame=single,
    breaklines=true,
    showstringspaces=false}

\begin{lstlisting}
# This is a comment
def hello_world():
    print("Hello, World!")  # Print a string
\end{lstlisting}
```

This is how the LaTeX code from above looks when compiled:

```
1 # This is a comment
2 def hello_world():
3     print("Hello, World!")  # Print a string
```

Explanation:

- **basicstyle=\ttfamily\footnotesize**: Sets the basic style for the code (monospaced and small).
- **keywordstyle=\color{blue}**: Sets the style for keywords (blue color).
- **stringstyle=\color{red}**: Sets the style for strings (red color).
- **commentstyle=\color{lightgray}**: Sets the style for comments (lightgray color).

- **backgroundcolor=\color{white}**: Sets the background color of the code block (white).
- **numbers=left**: Adds line numbers to the left of the code.
- **numberstyle=\tiny\color{gray}**: Sets the style for line numbers (tiny and gray).
- **stepnumber=1**: Numbers every line.
- **frame=single**: Adds a frame around the code block.
- **breaklines=true:** Allows long lines to break.

1.5.5 Including External Code Files

You can also include code from an external file using `\lstinputlisting`:

```
\lstinputlisting[language=Python]{path/to/your/code.py}
```

This command includes the content of the external file code.py with Python syntax highlighting.

1.6 Citations and Bibliography

You can manage your citations and bibliographies by using the `bibliography` and `bibtex` commands.

1.6.1 Including Citations in Your Document

This subsection will show you how to add citations in your LaTeX document.

First, you need to create a `.bib` file, containing your bibliography entries, i.e. your sources. Bib-entries are written in BibTeX format and might look like this:

```
@typeofmedia{key,
  title={Title of cited word},
  author={Author1, Author2, et al.},
  year={Year in which work was published},
  publisher={Name of Publisher}
}
```

Explanation:

- `\cite{key}`: This command is used to cite a source from your Bib-file.
- `key`: is the keyword which you give your citation to reference it in your text.
- `typeofmedia`: This should be replaced with the actual type of media your source is(book, article, journal, etc.)
- Fields like `title, author, year`, and `publisher` describe the source and have to be filled accordingly.

Depending on your citation style, (which can be set with the command `\bibliographystyle{}` in the `main.tex` file, the citations in your text might look like this: [2].
The bibliography is automatically added by using the command `\bibliographyfile.bib` in the `main.tex` file.
As an example, we provided a Bib-file called `sample.bib`, which contains example entries in different forms. Instead of typing the bibliography file entries manually, you can usually automatically get them from major publication search engines like Google Scholar or IEEE Xplore. For both of them, just click on the *cite* button, select *BibTeX* and copy the entry to your bibliography file.

1.7 Glossary

A glossary is a section at the end of a written work that defines confusing, technical, or advanced words. It is not mandatory to include a glossary in your thesis, but if you want to include one, here is how.
First, you create a new `tex`-file in which you write your glossary entries. In LaTeX you can define a glossary entry the following way:

```
\newglossaryentry{latex}{
    name=LaTeX,
    description={A document preparation system for high-quality typesetting}
}
```

Explanation:

- **label**: The first parameter is the label of this term and is used to reference it within the document with the gls command. In the example from above the label is 'latex'.
- **name**: The name parameter includes the word to be defined, in this case, 'LaTeX'.
- **description**: Includes the definition of the current term.

You can also define acronyms you use in your thesis. To do this, you open the `tex`-file containing your glossary entries and then define your acronym the following way:

```
\newacronym{rms}{RMS}{Root Mean Squared}
```

The structure is the same as of the `\newglossaryentry`. Here, the **label** is `rms`, the **name** is `RMS` and the **description** is `Root Mean Squared`.

To include the `glossary.tex` file in your thesis, you use the command `\input{glossary_filename}`. Here, you have to replace the `glossary_filename` parameter with the actual filename of the `tex`-file containing your glossary. This is already defined in the `main.tex` file in line 56.

To print your glossary you have to use the command `\printglossary` at the position in your thesis at which you want the glossary to be. We inserted it at the beginning of the thesis (before the table of contents).
If you have acronyms that you want to include, you use an additional keyword with the previous command: `\printglossary[type=\acronymtype]`.

1.7.1 Use Glossary entries in the text

After you have defined the terms, to use them while you are typing your LaTeX file use one of the commands described below:

- `\gls{ }`: This prints the term in lowercase.
  Example: `\gls{maths}` prints 'mathematics' when used.
- `\Gls{ }`: This prints the term lowercase with the first letter printed in uppercase.
  Example: `\Gls{maths}` prints 'Mathematics'.
- `\glspl{ }`: This prints the term in lowercase and puts it in its plural form.
  Example: `\glspl{formula}` prints 'formulas'.
- `\Glspl{ }`: This prints the term with the first letter in uppercase and puts it in its plural form.
  Example: `\Glspl{formula}` prints 'Formulas'.

Including your terms in your text, using the commands from above may look something like this:
In this thesis, we will use LaTeX to prepare a high-quality document. The LaTeX typesetting markup language is especially suitable for documents that include mathematics. Formulas are rendered easily once one gets used to the commands.

2 Modelling

2.1 Interior Magnet PMSM

This leads to the to the continuous time state space model of Equation (2.1).

$$\frac{d}{dt}\begin{bmatrix} i_{Sd} \\ i_{Sq} \end{bmatrix} = \begin{bmatrix} -\frac{R_S}{L_d} & \frac{L_q}{L_d}\omega_e(t) \\ -\frac{L_d}{L_q}\omega_e(t) & -\frac{R_S}{L_q} \end{bmatrix} \begin{bmatrix} i_{Sd} \\ i_{Sq} \end{bmatrix} + \begin{bmatrix} \frac{1}{L_d} & 0 \\ 0 & \frac{1}{L_q} \end{bmatrix} \begin{bmatrix} v_{Sd} \\ v_{Sq} \end{bmatrix} + \begin{bmatrix} 0 \\ -\frac{\psi_{PM}}{L_q}\omega_e(t) \end{bmatrix} \tag{2.1}$$

2.2 Coupling Dynamics

$$\begin{aligned} J_1 \frac{\mathrm{d}(\omega_{m1})}{\mathrm{d}t} &= T_e - c \cdot \Delta\varphi - k \cdot \Delta\omega_m \\ J_2 \frac{\mathrm{d}(\omega_{m2})}{\mathrm{d}t} &= T_l + c \cdot \Delta\varphi + k \cdot \Delta\omega_m \\ \frac{d(\varphi_{m1})}{dt} &= \omega_{m1} \\ \frac{d(\varphi_{m2})}{dt} &= \omega_{m2} \end{aligned} \tag{2.2}$$

3

Model Predictive Current Control

3.1 Prediction

Only Predict current dynamics, dont use coupling in prediction

3.1.1 Choice of adequate State Space Model

As derived in Section 2.1, the model of the interior magnet PMSM is given by Equation (3.1).

$$\underbrace{\frac{\mathrm{d}}{\mathrm{d}t}\begin{bmatrix} i_{Sd} \\ i_{Sq} \end{bmatrix}}_{\dot{\boldsymbol{x}}_c} = \underbrace{\begin{bmatrix} -\frac{R_S}{L_d} & \frac{L_q}{L_d}\omega_e(t) \\ -\frac{L_d}{L_q}\omega_e(t) & -\frac{R_S}{L_q} \end{bmatrix}}_{\boldsymbol{A}_c(\omega(t))} \underbrace{\begin{bmatrix} i_{Sd} \\ i_{Sq} \end{bmatrix}}_{\boldsymbol{x}_c} + \underbrace{\begin{bmatrix} \frac{1}{L_d} & 0 \\ 0 & \frac{1}{L_q} \end{bmatrix}}_{\boldsymbol{B}_c} \underbrace{\begin{bmatrix} u_{Sd} \\ u_{Sq} \end{bmatrix}}_{\boldsymbol{u}_c} + \underbrace{\begin{bmatrix} 0 \\ -\frac{\psi_{PM}}{L_q}\omega_e(t) \end{bmatrix}}_{\boldsymbol{E}_c(\omega(t))} \tag{3.1}$$

The nonlinear continuous time dynamic system is then given as

$$\dot{\boldsymbol{x}}_c = \boldsymbol{A}_c(\omega(t))\boldsymbol{x}_c + \boldsymbol{B}_c\boldsymbol{u}_c + \boldsymbol{E}(\omega(t)$$

or in compact notation:

$$\Sigma : (\boldsymbol{A}_c(\omega(t)), \boldsymbol{B}_c, \boldsymbol{E}(\omega(t)))$$

For most applications it is beneficial to work with a linear model, since this allows for the use of linear control approaches. However, the affine term regarding the induced voltage $\boldsymbol{E}(\omega(t))$ is non-negligable for the current dynamics $\dot{\boldsymbol{x}}$ at every operating point except standstill.
Using classical field-oriented PI current control, this is solved by splitting the system up into two linear, decoupled subsystems and two coupling terms (with the induced voltage

acting affine to one of the).

$$\dot{i_{Sd}} = \underbrace{\frac{1}{L_d} v_{Sd} - \frac{R_S}{L_d} i_{Sd}}_{\Sigma_{\text{d,decoupled}}} + \underbrace{\frac{L_q}{L_d} \omega_e i_{Sq}}_{\Sigma_{\text{d,coupled}}}$$

$$\dot{i_{Sq}} = \underbrace{\frac{1}{L_q} v_{Sq} - \frac{R_S}{L_q} i_{Sq}}_{\Sigma_{\text{q,decoupled}}} - \underbrace{\frac{L_d}{L_q} \omega_e i_{Sd}}_{\Sigma_{\text{q,coupled}}} - \underbrace{\frac{1}{L_q} \omega_e \psi_{PM}}_{\Sigma_{\text{q,affine}}}$$

The linear systems $\Sigma_{\text{d,decoupled}}$ and $\Sigma_{\text{q,decoupled}}$ are controlled using two independent PI controllers. The remaining terms are compensated by a feed-forward system.

Analyzing $\Sigma : (\boldsymbol{A}_c(\omega(t)), \boldsymbol{B}_c)$ in the context of linear parameter-varying systems, the framework of LPV System according to one has to quantify the influence of the parameter ω. Due to the balance of angular momentum of the motor shaft (2.2), the electrical rotor speed $\omega = \omega_e = \omega_{m1} p$ is Lipschitz-continuous. With the eigenvalues of the respective dynamical system (2.2) being slower than (3.1).

Thus, it was decided to use the affine prediction of Equation (3.1) for the prediction.

Bibliography

[1] "Overleaf documentation," https://www.overleaf.com/learn, accessed: 2024-05-22.

[2] D. E. Knuth, *The TeX book*. Addison-Wesley, 1984.

List of Figures

List of Tables

A

Appendix Title

This is an example of an appendix.